

Botnet Detection in IoT Devices Using Artificial Neural Network

Ahren Jun Sukirya
Department of Computer Science
Bina Nusantara University
Jakarta, Indonesia
Email: ahren.js@gmail.com

Gredion Prajena
Department of Computer Science
Bina Nusantara University
Jakarta, Indonesia
Email: gredionprajena@binus.ac.id

Abstract—The rapid growth of Internet of Things (IoT) devices in numerous industries has increased the risks associated with security flaws and data integrity concerns. Detecting malware in IoT devices, which is crucial for identifying malicious activity or faulty operations, is a significant research topic. This paper investigates the use of Artificial Neural Networks (ANNs) in IoT botnet detection, utilizing neural networks' adaptive learning capabilities to discover deviations from expected device behavior. We explain how to accurately discern between normal and abnormal behavior using ANN models trained on a labeled dataset of benign and malicious network traffic. This research analyzes the accuracy, precision, recall, and F1-score metrics of ANNs in detecting botnets to improve IoT network protection mechanisms. While the proposed ANN model achieves high detection accuracy, its performance is evaluated solely on the IoT-23 dataset. Future work will focus on testing across multiple IoT datasets to improve generalizability.

Keywords: *Internet of Things (IoT), Artificial Neural Network (ANN), Cybersecurity, Machine Learning, Network Intrusion Detection, IoT Security*

I. INTRODUCTION

The Internet of Things (IoT) ecosystem connects millions of devices worldwide, allowing for better conditions in healthcare, industry, transportation, and residences. However, increased accessibility raises the potential for cyber attacks, in which attackers exploit flaws to influence IoT equipment. Malware detection, particularly for botnets in IoT, is critical for recognizing infected devices. Traditional security mechanisms frequently fail in IoT networks due to low processing power and the large variety of devices. As a result, machine learning techniques, particularly Artificial Neural Networks (ANNs), have developed as effective tools for detecting infected IoT behaviors.

Artificial Neural Networks (ANNs) have the potential to learn complex patterns in data, making them ideal for IoT botnet detection. The goal of this study is to assess the effectiveness of ANNs for botnet identification by training models on a labeled dataset containing a variety of benign and malicious behaviors. We investigate the resilience of ANNs in discerning between normal and malicious traffic patterns, focusing on their adaptability and accuracy in detecting botnets.

II. LITERATURE REVIEW

2.1 IoT Security Challenges

IoT devices are vulnerable due to weak authentication mechanisms, insufficient firmware updates, and limited computational capacity [1], [2]. These weaknesses make them prime targets for botnet attacks, necessitating lightweight yet effective detection mechanisms.

2.2 Botnet Types and Techniques

IoT botnets, such as Mirai and Torii, exploit weak credentials and unpatched vulnerabilities to compromise devices [2]. These botnets often use encrypted communications and dynamic traffic patterns, making detection challenging with traditional methods.

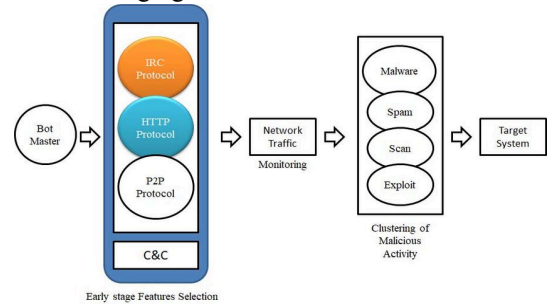


Fig. 1. Botnet Architecture

2.3 Traditional Botnet Detection Techniques

Traditional botnet detection relies on signature-based and anomaly-based techniques. Signature-based methods identify known patterns but fail against evolving botnets [3], while anomaly-based approaches detect new threats but suffer from high false-positive rates in heterogeneous IoT traffic [4]. These limitations motivate machine learning and deep learning solutions.

2.4 Machine Learning-Based Detection Approaches

Machine learning techniques, including SVM, Random Forest, and KNN, have shown promising results [9], but they struggle to generalize to unseen attacks. Deep learning approaches, such as CNNs [10] and LSTMs [11], achieve higher accuracy but require significant computational resources. In contrast, ANNs provide a balance between detection performance and efficiency, making them suitable for IoT scenarios with limited resources.

2.5 Hybrid and Transfer Learning Models

Recent studies explore hybrid models combining machine learning and deep learning, achieving improved detection rates [15]. Similarly, transfer learning has been applied to adapt pretrained CNNs for IoT botnet detection, improving performance on unseen botnet variants [16].

2.6 Summary and Research Gap

While CNNs [10] and LSTMs [11] provide higher accuracy, they demand significant computational resources unsuitable for IoT edge environments. Hybrid [15] and transfer learning models [16] also achieve strong results but introduce additional complexity. We therefore propose a lightweight feedforward ANN that balances detection accuracy and efficiency, making it well-suited for constrained IoT deployments.

III. METHODOLOGY

3.1 Data Collection

This study employs the IoT-23 dataset, a publicly available collection of labeled network traffic created by the Stratosphere Laboratory, Czech Technical University [20]. The dataset contains network captures from 20 IoT scenarios involving both benign and malicious activity. Malicious traffic includes various botnets such as Mirai, Gafgyt, Torii, Hajime, and others. Each capture was manually verified and annotated by the dataset authors, ensuring reliability for supervised learning. The IoT-23 dataset provides over 12 categories of network events, enabling multi-class classification of botnet types. It was selected for its comprehensive coverage of real-world IoT botnet behavior and its accessibility for academic research.

3.2 Data Preprocessing

Before training machine learning models, the raw dataset undergoes several preprocessing steps to ensure that the data is clean, normalized, and appropriately formatted for input into the model. The following preprocessing techniques were applied to the IoT-23 dataset:

- Label Encoding

The dataset contains a categorical label column, which indicates the nature of the botnet traffic. Since most machine learning algorithms, including deep learning models, require numerical input, this label is encoded into numeric values. Each botnet family and benign class was assigned a numeric label (0-11) and one-hot encoded for multi-class training using scikit-learn utilities, ensuring compatibility with neural network inputs and avoiding artificial ordinal relationships.

- Normalization

Numerical features were normalized using StandardScaler to ensure zero mean and unit variance, preventing any single feature from dominating the learning process. Additionally, a VarianceThreshold of 0.01 was applied to remove low-variance features and improve computational efficiency. This helps center features around zero mean and unit variance, which improves ANN training stability.

The features selected from the dataset include statistical and flow-based attributes such as packet count, packet size, protocol type, duration, source and destination byte rate, and connection state. These features were chosen because they capture traffic volume, communication behavior, and temporal dynamics that distinguish benign activity from botnet behavior, while redundant or non-informative fields (e.g., timestamps, IP addresses) were excluded.

- Resampling:

Due to significant class imbalance in IoT-23, Random Oversampling (ROS) was applied to duplicate minority-class samples and balance the dataset. While more advanced techniques like SMOTE [12] or GAN-based augmentation [13] could produce synthetic diversity, ROS was chosen due to hardware constraints and to avoid the additional computational overhead introduced by complex augmentation.

This resampling is applied only to the training data, ensuring that the model does not overfit or become biased due to the class imbalance. After resampling, the class distribution is rechecked to confirm that the training data is now balanced.

- Train-Test Split

The final preprocessing step involves splitting the dataset into training and testing subsets using a fixed random seed to ensure reproducibility. This is done to ensure that the model can be trained on one portion of the data and evaluated on an unseen portion. The dataset is split into 80% training and 20% testing using the `train_test_split` function from scikit-learn. The training set is used to train the machine learning model, while the testing set is used to evaluate its performance.

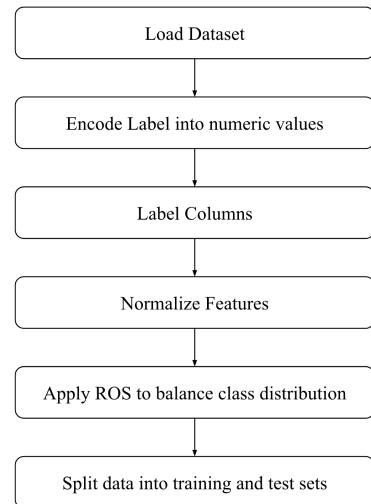


Fig. 2. Data Preprocessing Diagram

3.3 Model Architecture

The model employed for classifying IoT traffic data is a feedforward artificial neural network (ANN), which is specifically designed to categorize network traffic into 12

(twelve) distinct classes, one for each botnet. We selected a feedforward ANN due to its efficiency in handling structured IoT traffic data and its suitability for resource-constrained environments [10], [11], [14]. Unlike CNNs, which are optimized for spatial feature extraction [10], and LSTMs, which excel in temporal sequence modeling [11], ANNs offer comparable detection performance with lower computational cost [14], making them better suited for IoT scenarios. The architecture of the network is structured to efficiently learn from the features of the traffic data while addressing the challenge of class imbalance and overfitting. Below is a detailed description of the architecture and its key components:

- **Input Layer**

The input layer is the first component of the neural network, where data is fed into the model. This layer consists of 512 neurons, which correspond to the number of features in the processed dataset. These features represent the various attributes of the IoT traffic, such as packet size, protocol type, duration, source and destination IP addresses, and more. The input layer serves as the entry point for the raw traffic data, which is preprocessed and normalized to ensure that the neural network can effectively learn patterns from the data.

- **Hidden Layers**

The network contains three fully connected layers with 512, 256, and 128 neurons, respectively. This architecture was selected after preliminary experiments comparing smaller (128-64-32) and deeper (1024-512-256-128) configurations, where the chosen structure offered the best trade-off between detection accuracy and computational efficiency. Each of these hidden layers employs the Rectified Linear Unit (ReLU) activation function, which introduces non-linearity into the model. ReLU was chosen for faster convergence and to mitigate vanishing gradients, enabling faster training and better performance when handling deep neural networks. By using ReLU, the model can better model complex and intricate patterns in the IoT traffic data.

- **Dropout Layers**

To combat the issue of overfitting, especially given the relatively large number of parameters in the hidden layers, Batch Normalization and dropout layers are incorporated after each hidden layer. The dropout rate is set to 0.3, meaning that 30% of the neurons are randomly "dropped out" during each training iteration. This forces the model to learn more robust features by preventing it from becoming overly reliant on any single neuron, which helps improve the model's generalization to unseen data.

- **Output Layer**

The output layer of the network consists of 12 neurons, each representing one of the possible classes. The softmax activation function is applied in this layer, which ensures that the output values are scaled to lie between 0 and 1 and sum up

to 1. This conversion enables the model to produce class probabilities, where the class with the highest probability is selected as the model's prediction. The softmax function is particularly useful for multi-class classification tasks, as it allows the network to estimate the likelihood of each class based on the features of the input data.

- **Regularization**

To prevent overfitting and improve the model's ability to generalize, L2 regularization is applied to all hidden layers. L2 regularization, also known as weight decay, penalizes large weights by adding a term to the loss function that is proportional to the squared sum of the weights. This discourages the network from relying too heavily on any particular feature, which can lead to better generalization of unseen data and improved performance on real-world traffic datasets.

- **Optimizer**

The model uses the Adam optimizer with a learning rate of 0.0001 to minimize the loss function during training. Adam is a popular optimization algorithm because it adapts the learning rate for each parameter based on estimates of the first and second moments of the gradients. This makes it well-suited for problems with noisy gradients or sparse data. The small learning rate of 0.0001 ensures that the model converges gradually and avoids overshooting the optimal solution during training.

- **Loss Function**

The model utilizes the categorical cross-entropy loss function, which is commonly used in multi-class classification tasks. This loss function computes the difference between the predicted class probabilities (from the softmax output) and the true class labels to minimize this difference. Categorical cross-entropy is effective in this context because it penalizes the model more heavily when it makes confident, incorrect predictions, encouraging it to make more accurate classifications.

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	16,384
batch_normalization (BatchNormalization)	(None, 512)	2,048
dropout (Dropout)	(None, 512)	0
dense_1 (Dense)	(None, 256)	131,328
batch_normalization_1 (BatchNormalization)	(None, 256)	1,024
dropout_1 (Dropout)	(None, 256)	0
dense_2 (Dense)	(None, 128)	32,896

batch_normalization_2 (BatchNormalization)	(None, 128)	512
dropout_2 (Dropout)	(None, 128)	0
dense_3 (Dense)	(None, 12)	1,548

Table 1. Sequential model architecture for this research

3.4 Model Training

The model was implemented using TensorFlow and Keras, conducted on Windows operating system, with scikit-learn used for preprocessing and Matplotlib for visualization. The model was trained with a batch size of 64 using the Adam optimizer (learning rate = 0.0001) for a maximum of 100 epochs. Early stopping with a patience of 10 epochs was employed to halt training if no improvement in validation loss was observed. This helps in saving computational resources by avoiding unnecessary epochs once the model has reached its optimal performance. These hyperparameters were selected empirically after preliminary experiments demonstrated stable convergence. All training sets were also run with fixed random seeds to ensure reproducible results.

To address the inherent class imbalance in the dataset, class weights are calculated using the `compute_class_weight` function from the `sklearn.utils.class_weight` module. The class weights are computed based on the distribution of the target labels in the training set, ensuring that the model gives more importance to the minority classes. This adjustment allows the model to make more accurate predictions for the underrepresented classes and reduces the risk of the model being biased towards the majority class. The computed class weights are then passed as a parameter during the training process, allowing the model to prioritize the correct classification of minority class instances while still learning the characteristics of the majority class.

Throughout the training process, the model is validated on a separate test set, which is crucial for assessing its generalization ability and ensuring that it performs well on unseen data. The training history is logged, capturing key metrics such as loss, accuracy, precision, and recall at each epoch. This history is analyzed post-training to evaluate the model's learning curve, identify any signs of overfitting or underfitting, and make any necessary adjustments to the model or training parameters.

Additionally, the training setup allows for adjustments in hyperparameters, such as the learning rate and the architecture of the neural network, to optimize the model's performance. The implementation and dataset preprocessing scripts are available on GitHub [19].

3.5 Evaluation Metrics

The model employs several evaluation metrics to assess its performance in a multi-class classification setting:

- **Accuracy** measures overall correctness or correct predictions (both benign and malicious) made by the model out of the total predictions.
- **Precision** measures the proportion of true positive predictions out of all instances predicted as positive by the model. In the context of botnet detection, it indicates how many of the traffic samples classified as malicious are actually malicious.

- **Recall** evaluates the proportion of true positive predictions out of all actual positive instances in the dataset. For botnet detection, recall measures how well the model identifies all the instances of malicious traffic.
- **F1-score** is the harmonic mean of precision and recall and provides a balanced measure between the two. The F1-score ensures that the model does not heavily favor one metric over the other, making it a more reliable evaluation metric in cases where both false positives and false negatives carry significant consequences.

3.6 Performance Evaluation

The performance of the trained model is assessed through the evaluation metrics to ensure its robustness and reliability in detecting both benign and malicious traffic in IoT devices. A confusion matrix is also used to provide a visual representation of the actual versus predicted classifications, showing the true positives (TP), true negatives (TN), false positives (FP), and false negatives (FN) for each class.

The confusion matrix offers detailed insights into how well the model distinguishes between benign and malicious traffic and whether it is prone to misclassifying one type of traffic as another. By analyzing the matrix, one can determine if there are particular classes where the model performs poorly, such as frequently misclassifying benign traffic as Mirai botnet traffic, or vice versa.

IV. RESULTS

Final evaluation was conducted on the untouched test set using the specified metrics. The test set was not subjected to oversampling, preserving real-world class distribution. The model achieved high classification results for all classes.

Accuracy	Precision	Recall	F1-Score
99.78%	99.75%	99.74%	99.75%

Table 2. Overall performance results

The confusion matrix for the model on the test set is as follows:

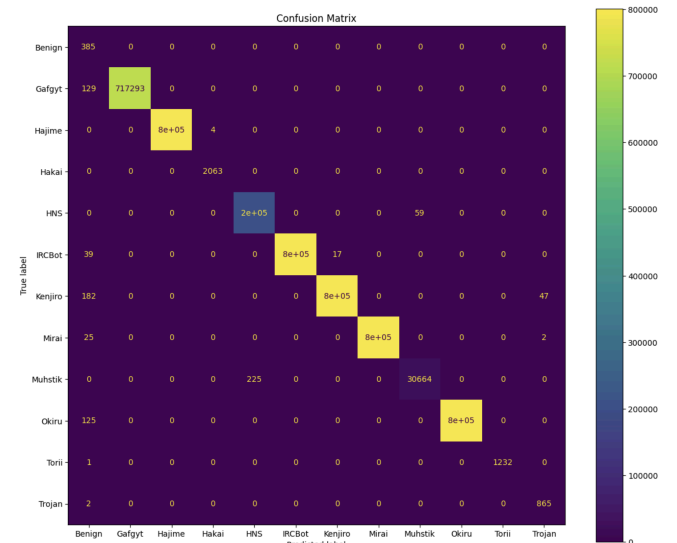


Fig. 3. Confusion matrix for the evaluation

From the confusion matrix, we observe that Mirai and Okiru classes have occasional misclassifications due to similar network traffic patterns. Conversely, Hajime and Torii are detected with near-perfect accuracy, demonstrating that the model learns distinct features for advanced botnets.

Overall, the model has demonstrated high precision, meaning it accurately identified botnet activities with minimal false positives. Similarly, the recall score indicates that the model was able to correctly detect nearly all instances of botnet activity, minimizing false negatives. The balanced F1-score confirms the robustness of the model in handling the trade-off between precision and recall, ensuring reliable detection across all classes. The results are based on a single training run. To ensure statistical robustness, future experiments will include repeated runs with different random seeds and report mean and standard deviation for accuracy, precision, recall, and F1-score.

V. DISCUSSION

To evaluate the relative performance of the proposed Artificial Neural Network (ANN) model, the results are compared with those reported in existing literature. The table below presents a summary of various machine learning and deep learning-based approaches for IoT botnet detection, along with their reported performance metrics. Where applicable, reported results are based on different datasets. Thus, direct comparison should be interpreted cautiously.

Study	Approach	Accuracy	Notes
This research	Feedforward ANN	~99.78%	Uses IoT-23 dataset with oversampling and class balancing.
Yang et al. (2019) [9]	Random Forest	~98.2%	Good with known attacks, struggles with unseen variants.
Chen et al. (2020) [11]	LSTM (Deep Learning)	~99.2%	Effective in learning temporal patterns in IoT traffic.
Xu et al. (2020) [10]	CNN	~99.4%	Automatically learns spatial features from raw traffic data.
Kang et al. (2019) [14]	Multi-Layer Perceptron (ANN)	~98.6%	Demonstrated strong detection, but less adaptable across device types.

Wang et al. (2021) [16]	Transfer Learning (CNN-based)	~99.5%	Improved adaptability to unseen variants.
Kim et al. (2020) [15]	Hybrid: Random Forest + DNN	~99.6%	Leveraged feature selection + DL classifier for high performance.
Shareef et al. (2024) [13]	Dual-channel GAN + Zebra Optimization	~99.8%	One of the highest performing methods using ensemble & optimization.
Ullah & Mahmoud (2022) [14]	Feedforward Neural Network	~98.3%	Used flow and flag features from traffic logs.
Abu Al-Haija & Al-Dala'ien (2022) [18]	Ensemble Learning (ELBA-IoT)	~98.9%	Boosted learning model combining multiple classifiers.

Table 3. Performance comparison with other models

The ANN model achieved high performance with precision, recall, and an F1-score of over 99.7% for all classes, indicating that the model was able to accurately classify benign and botnet traffic with minimal false positives or false negatives. The resampling techniques and the class weights used during the training helped ensure that the model was not biased toward the majority classes. The balanced class distribution in the training set, combined with the model's evaluation metrics, demonstrates that the model can effectively detect botnet traffic even in imbalanced datasets.

VI. CONCLUSION

The ANN-based model developed in this study offers an excellent solution for detecting botnet activity in IoT networks. The model achieved near-perfect results, demonstrating the ANN's robustness and reliability in identifying botnet attacks, making it a viable tool for real-world IoT security applications. However, the model has not yet been tested in real-time or production environments, which are essential to fully assess its practical utility and robustness in dynamic, noisy network conditions.

VII. FUTURE WORK

While the proposed ANN model demonstrated exceptional performance on the IoT-23 dataset, several limitations remain that present opportunities for further research and development. Due to hardware constraints,

specifically a laptop with 16 GB of RAM and limited storage capacity, only approximately 3.4 GB of the IoT-23 dataset could be utilized in this study. Attempting to process larger portions caused system crashes around the 20th epoch of training. Consequently, the evaluation was limited to a single dataset. Future work will address this limitation by expanding the experimental evaluation to include additional benchmark datasets such as UNSW-NB15 and CICIDS2017, as well as real-world IoT traffic captures. These efforts will improve the generalizability and robustness of the model across diverse network environments.

To strengthen scientific rigor, multiple training runs with different random seeds will be conducted to measure the mean and standard deviation of key performance metrics (accuracy, precision, recall, and F1-score). This will provide statistical validation and ensure that the reported performance is consistent across runs. Furthermore, although the ANN achieved high accuracy on the IoT-23 dataset, its ability to detect unseen or evolving botnet families has not yet been tested. Future studies will explore domain adaptation and transfer learning techniques to improve generalization toward novel attack types and traffic patterns not represented in the training data.

From a deployment perspective, further investigation is required to optimize the ANN for resource-constrained IoT and edge devices. Techniques such as model pruning, weight quantization, and TinyML conversion (e.g., TensorFlow Lite) will be explored to reduce the model's size and latency while maintaining detection performance. These optimizations could enable real-time inference on low-power embedded systems. Finally, future work will include computational cost profiling, detailing the model's training time, inference speed, and energy consumption under different configurations. This analysis will help assess the trade-offs between performance and efficiency, providing valuable insight for practical deployment in real-world IoT environments.

ACKNOWLEDGEMENT

Data Availability Statement

The dataset used in this study is available at the Stratosphere Laboratory: <https://www.stratosphereips.org/datasets-iot23>. The source code is available at the Github Repository: <https://github.com/AhrenJS/IoT-Botnet-Detection>.

Authorship Contribution Statement

Ahren Jun Sukirya: Conceptualization, Methodology, Software, Resources, Writing - Original Draft, Project Administration.

Gredion Prajena: Conceptualization, Validation, Writing - Review & Editing, Supervision.

REFERENCES

- [1] J. Shin, K. Park, and Y. Lee, "Security challenges of IoT devices: A comprehensive survey," *IEEE Internet of Things Journal*, vol. 7, no. 8, pp. 7752-7771, Aug. 2020.
- [2] Z. Zhao, H. Li, and L. Wu, "Understanding IoT botnets: Characteristics, detection, and challenges," *IEEE Access*, vol. 7, pp. 121-135, 2019.
- [3] H. Alasmay, A. Abusnaina, and D. Mohaisen, "Analyzing and detecting emerging Internet of Things malware: A graph-based approach," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8977-8988, Oct. 2019.
- [4] H. HaddadPajouh, A. Dehghantanha, R. Khayami, and K.-K. R. Choo, "A deep recurrent neural network-based approach for Internet of Things malware threat hunting," *Future Generation Computer Systems*, vol. 85, pp. 88-96, Aug. 2018.
- [5] B. Sharma, R. Kumar, A. Kumar, M. Chhabra, and S. Chaturvedi, "A systematic review of IoT malware detection using machine learning," in *Proc. 2023 10th Int. Conf. Computing for Sustainable Global Development (INDIACom)*, New Delhi, India, Mar. 2023, pp. 91-96.
- [6] M. Shobana and S. Poonkuzhali, "A novel approach to detect IoT malware by system calls using deep learning techniques," in *Proc. Int. Conf. Innovative Trends in Information Technology (ICITIIT)*, Kottayam, India, Feb. 2020, pp. 1-5.
- [7] C. Tamás, D. Papp, and L. Buttyán, "SIMBioTA: Similarity-based malware detection on IoT devices," in *Proc. 6th Int. Conf. Internet of Things, Big Data and Security (IoTBDs)*, Apr. 2021, pp. 58-69.
- [8] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cybersecurity intrusion detection," *IEEE Commun. Surveys Tuts.*, vol. 18, no. 2, pp. 1153-1176, 2nd Quart. 2016.
- [9] A. Yang, M. Park, and S. Kim, "Botnet detection in IoT using random forest classifiers," in *Proc. IEEE Int. Conf. Consumer Electronics (ICCE)*, Jan. 2019, pp. 1-5.
- [10] J. Xu, L. Wang, and K. Zhao, "CNN-based IoT botnet detection," *IEEE Trans. Netw. Serv. Manag.*, vol. 17, no. 4, pp. 2341-2354, Dec. 2020.
- [11] Y. Chen, H. Huang, and W. Lin, "Botnet detection using LSTM-based deep learning models," *IEEE Access*, vol. 8, pp. 132-145, 2020.
- [12] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321-357, 2002.
- [13] S. Shareef, R. Chaitanya, and S. Chennupalli, "Enhanced botnet detection in IoT networks using zebra optimization and dual-channel GAN classification," *Sci. Rep.*, vol. 14, no. 17148, pp. 1-12, 2024.
- [14] I. Ullah and Q. Mahmoud, "An anomaly detection model for IoT networks based on flow and flag features using a feed-forward neural network," in *Proc. Int. Conf. Communications and Network Security*, Nov. 2022, pp. 363-368.
- [15] A. Kim, H. Lee, and S. Cho, "Hybrid botnet detection using random forest and deep learning classifiers," *IEEE Internet Things J.*, vol. 8, no. 12, pp. 9871-9885, Jun. 2021.
- [16] Y. Wang, J. Li, and T. Sun, "Transfer learning-based IoT botnet detection with CNN," in *Proc. IEEE Global Communications Conference (GLOBECOM)*, Dec. 2021, pp. 1-6.
- [17] N. Moustafa and J. Slay, "The evaluation of network anomaly detection systems: Statistical analysis of the UNSW-NB15 dataset and comparison with the KDD99 dataset," in *Proc. IEEE Int. Conf. Security and Privacy (SP)*, Mar. 2016, pp. 1-14.
- [18] I. Sharafaldin, A. Habibi Lashkari, and A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. Int. Conf. Information Systems Security and Privacy (ICISSP)*, Jan. 2018, pp. 108-116.
- [19] A. J. Sukirya, "IoT Botnet Detection," GitHub repository, 2024. <https://github.com/AhrenJS/IoT-Botnet-Detection>
- [20] Stratosphere Laboratory, "IoT-23: A labeled dataset with malicious and benign IoT network traffic," Czech Technical University, 2020. <https://www.stratosphereips.org/datasets-iot23>
- [21] A. Sharma and D. Singh, "Lightweight TinyML models for IoT malware detection," *IEEE Access*, vol. 12, pp. 64321-64334, 2024.
- [22] L. Zhang et al., "Edge-optimized neural networks for IoT intrusion detection," *Sensors*, vol. 23, no. 2, pp. 1456-1470, 2023.